CAPSTONE PROJECT REPORT · VERSION 1.1

A Multi-Agent Voice-Based AI System for Hybrid Learning Using RAG

Retrieval-Augmented Generation · Multi-Agent Systems · Voice AI

Venkatraman Viswanathan
Post Graduate Program in Generative AI & Machine Learning · Infiltra Technologies
Original report April 2026 · Version 1.1 corrected October 2026

About version 1.1. This version corrects version 1.0 (April 2026) so that it documents the prototype as it was actually built. Version 1.0 remains available in the Zenodo version history. Changes:

- Routing is described as implemented: the mode is selected by whether a PDF is uploaded (a rule), not by an intent classifier.
- Voice is described as implemented: text input, with spoken output through pyttsx3 on the host computer. Voice input was not implemented.
- Tools are corrected to those in the code (pypdf, word-based chunking, all-MiniLM-L6-v2, FAISS, Llama 3 via Ollama, pyttsx3, Streamlit).
- The "30–40% improvement in response relevance" statement is removed: no experiment supported it.
- Percentages attributed to two unreferenced studies in section 6 are removed.
- Every feature is labelled **Implemented**, **Conceptual design** or **Planned**. The five-agent conceptual architecture is kept.

Abstract

This report presents a five-agent conceptual architecture for hybrid learning that combines Retrieval-Augmented Generation (RAG) over user-provided documents with general topic explanations and spoken output, together with a working prototype of that design. The prototype, built with Streamlit, FAISS, Sentence Transformers and Llama 3 (via Ollama), answers questions about an uploaded PDF using retrieved passages, explains general topics when no document is provided, generates study notes, and reads the answer aloud. In the prototype the mode is selected by a rule (whether a PDF is uploaded) and input is by text. Intent-based routing, speech input and in-browser audio are documented as planned enhancements. No experimental evaluation has yet been carried out; section 5 sets out the planned evaluation framework.

Keywords: Multi-Agent Systems, Retrieval-Augmented Generation, Hybrid Learning, Conversational AI, Voice-Based AI, Intelligent Tutoring Systems, LLM, FAISS, Vector Databases

1 Problem Statement

Many AI learning tools are limited by their architecture. Four gaps motivate this work:

Gap	Challenge	Consequence
Limited Adaptability	Fixed response pipelines cannot adjust to varying learning contexts or user expertise	One-size-fits-all responses
Knowledge Source Isolation	Systems constrained to either document repositories or external knowledge	Learners cannot easily switch between personal and general knowledge sources
No Conversational Interface	Text-only interactions	Less natural, less accessible learning experience
Static Routing Logic	All queries processed identically	The same retrieval strategy is applied regardless of query type

Table 1: Four problem areas — challenges and consequences

2 Contribution and Design Elements

This work proposes a multi-agent architecture for hybrid learning that combines document-centric retrieval with general topic explanation and spoken output, and implements a first prototype of it. The table separates what the prototype implements from what remains conceptual or planned.

#	Design element	Approach	Status in prototype
1	Multi-agent orchestration	Separate agents for retrieval, explanation, notes and voice, coordinated by a central component	**Implemented** as separate modules; coordination is rule-based
2	Hybrid learning framework	Document mode (RAG over an uploaded PDF) and Topic mode (LLM explanation) in one application	**Implemented**
3	Voice-based interaction	Spoken output of each answer	**Implemented** output (pyttsx3); voice input **Planned**
4	Intent-based routing	Classify each query as document, topic or mixed	**Planned**; prototype selects mode by PDF upload
5	Context-aware, multi-turn responses	Preserve conversation context across turns	**Planned**; each query is independent
6	Modular architecture	Loosely coupled agent modules with simple interfaces	**Implemented**

Table 2: Design elements and implementation status (corrected in v1.1)

3 System Architecture

Conceptual design. The Agentic Voice Tutor is designed as five agents: a Coordination Agent, a Document Retrieval Agent, a Topic Knowledge Agent, a Summarization Agent and a Voice Interaction Agent. Figure 1 shows this conceptual design. In the conceptual design the Coordination Agent classifies intent; in the prototype that role is performed by a rule (see below).

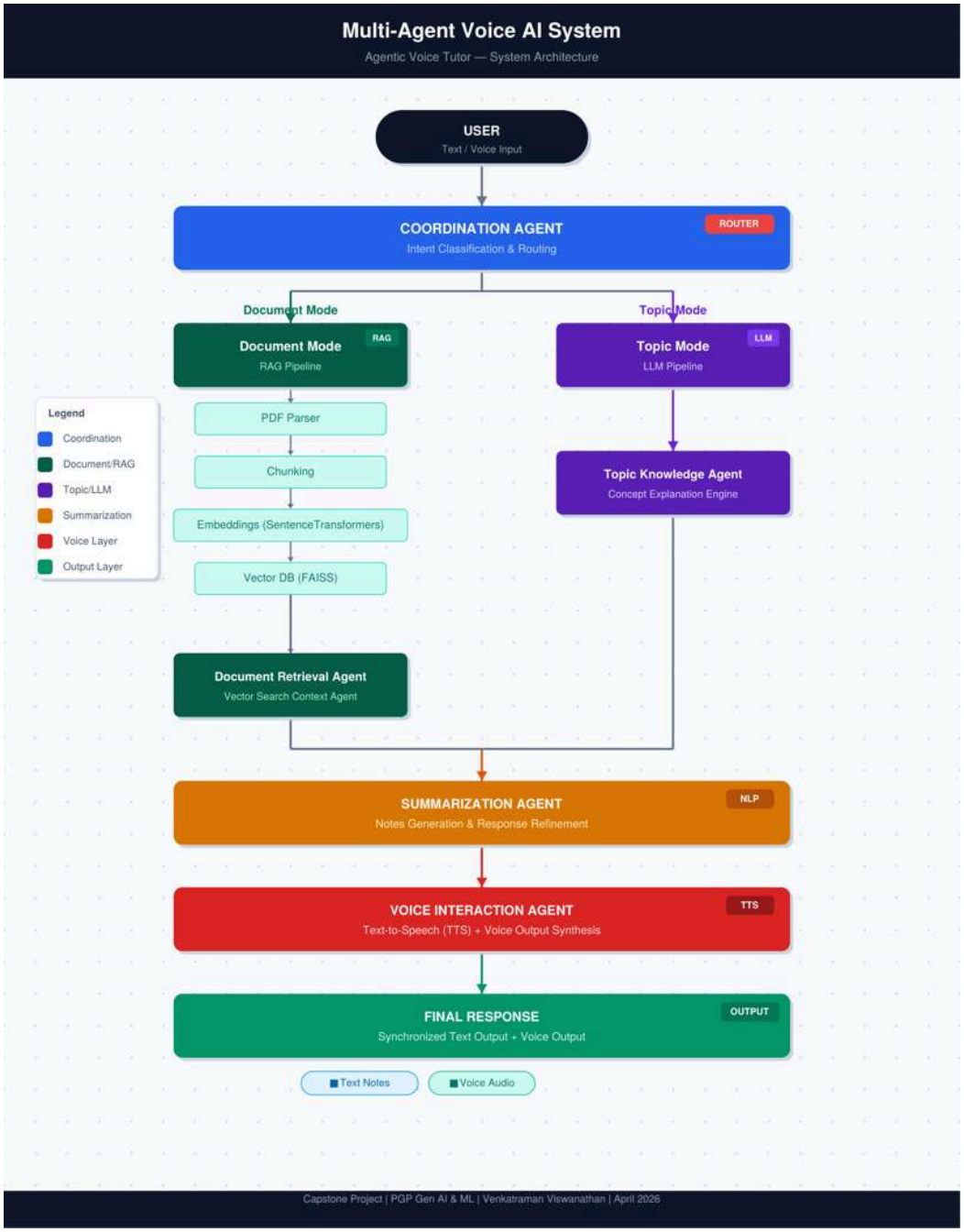


Figure 1: Conceptual five-agent architecture (design; see Tables 3–6 for what the prototype implements)

Prototype implementation (April 2026). The tables below list the components as implemented in the prototype source code.

Document Mode Pipeline (RAG) — as implemented

Stage	Component	Technology	Output
1	PDF Parser	pypdf (PdfReader)	Raw text extracted from the uploaded PDF
2	Chunking	Word-based splitter	Chunks of 500 words, no overlap
3	Embeddings	Sentence Transformers, all-MiniLM-L6-v2	Dense vector for each chunk
4	Vector Store	FAISS IndexFlatL2, built when a question is asked	Indexed chunk embeddings
5	Retrieval Agent	FAISS search, top-3 chunks	Context passed to the tutor agent
6	Tutor Agent	Llama 3 via Ollama	Explanation using the retrieved context

Topic Mode Pipeline (LLM) — as implemented

Stage	Component	Technology	Output
1	Mode selection	Rule: no PDF uploaded → Topic mode	Query sent to the tutor agent without document context
2	Tutor Agent	Llama 3 via Ollama	General explanation of the topic

Output Layer — as implemented

Component	Function	Technology	Delivery
Notes (Summarization) Agent	Summarizes the explanation into study notes	Llama 3 via Ollama	Notes shown under the explanation
Voice Interaction Agent	Reads the explanation aloud	pyttsx3 (offline text-to-speech)	Speech on the host computer's speakers
User Interface	PDF upload, text question, results	Streamlit	Explanation and notes on screen

Tables 3–5: Prototype implementation — Document mode, Topic mode and output layer

Agent inventory: conceptual role and prototype

Agent	Conceptual role	Prototype implementation	Status
Coordination	Classify intent and route queries	Rule: Document mode if a PDF is uploaded, otherwise Topic mode	**Rule-based** **LLM routing planned**
Document Retrieval	RAG over uploaded documents	pypdf, 500-word chunks, all-MiniLM-L6-v2, FAISS top-3	**Implemented**
Topic Knowledge	Open-domain explanations	Same Llama 3 tutor agent, without document context	**Implemented** (shared agent)
Summarization	Concise explanation and notes	Llama 3 "summarize" step	**Implemented**
Voice Interaction	Speech input and output, conversation	pyttsx3 output on the host; text input only	**Output** **Input planned**

Table 6: Five-agent inventory — conceptual role, prototype implementation and status

4 System Workflow (as implemented)

Step	Stage	Component	Processing detail
1	Query input	Streamlit UI	User types a question and optionally uploads a PDF
2	Mode selection	Coordination (rule)	PDF uploaded → Document mode; otherwise Topic mode
3	Retrieval (Document mode)	Retrieval agent	PDF parsed → chunked → embedded → FAISS top-3 chunks
4	Explanation	Tutor agent	Llama 3 generates the explanation, with retrieved context in Document mode
5	Notes	Notes agent	Llama 3 summarizes the explanation into notes
6	Output	UI and voice agent	Explanation and notes shown on screen; explanation spoken by pyttsx3

Table 7: Prototype workflow (corrected in v1.1)

5 Planned Evaluation Framework

No evaluation has yet been carried out on the prototype. The framework below sets out how the system is intended to be evaluated; the targets are design goals, not results.

Metric	Definition	Planned measurement	Design target
Response accuracy	Factual correctness of answers	Human evaluation against reference answers	>85% on a test query set
Context relevance	Fit between retrieved chunks and the question	Human or automated relevance rating	>80% relevance
System latency	Time from query to spoken output	Automated timing over 100 test queries	<3 seconds average
User satisfaction	Learner-reported experience	5-point Likert survey	>4.0 / 5.0
Learning effectiveness	Knowledge retention	Pre/post quiz comparison	Improvement in post-session scores

Table 8: Planned evaluation framework — design targets, not results

6 Potential Applications and Research Alignment

Application domain	Potential use	Relevant capability
AI-based tutoring	Explanations of course documents and topics	Document and Topic modes
Enterprise knowledge assistants	Q&A over internal documents for onboarding and training	Document mode (RAG) + notes
Corporate training and onboarding	Study notes from training material	Notes agent
Research and academic support	Explaining and summarizing papers	Document mode + notes
Accessibility	Spoken output for learners who prefer listening	Voice output

Table 9: Potential application domains (not yet deployed)

Research alignment

- **Retrieval-Augmented Generation:** grounding answers in retrieved passages follows the RAG approach of Lewis et al. [1].
- **Vector search:** FAISS [2] provides the similarity search over chunk embeddings.
- **Generative AI for learning:** the work addresses opportunities for LLMs in education discussed by Kasneci et al. [3].
- **AI-driven decision intelligence:** the modular multi-agent pattern connects enterprise AI work with educational technology.

7 Future Enhancements

Enhancement	Description
Intent-based routing	LLM-based classification of each query as document, topic or mixed, replacing the upload rule
Speech input	Speech recognition (for example Whisper) for spoken questions
In-browser audio	Audio played in the learner's browser instead of the host's speakers
Separate topic agent and mixed answers	A dedicated topic knowledge agent, and merged answers for questions that need both sources
Conversation context	Multi-turn memory across questions
Evaluation	Run the framework in section 5 and report measured results
Adaptive learning, skill-gap detection, RL feedback, multilingual voice	Longer-term research directions

Table 10: Planned enhancements

Conclusion

This project proposes a five-agent architecture for hybrid learning and delivers a working prototype that answers questions from uploaded PDFs using retrieval, explains general topics, generates notes and reads answers aloud. The prototype uses rule-based mode selection, text input and offline speech output. The main next steps are intent-based routing, speech input, in-browser audio and a measured evaluation of answer quality and latency.

References

1. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.
2. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.
3. E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learning and Individual Differences, vol. 103, article 102274, 2023.

About the Author

Venkatraman Viswanathan

Post Graduate Program in Generative AI & Machine Learning · Infiltra Technologies

Venkatraman Viswanathan is a researcher and technology leader working on AI-driven decision systems, multi-agent architectures and enterprise knowledge systems. This capstone project was completed as part of his Post Graduate Program in Generative AI and Machine Learning.

Submitted as a Capstone Project Report, April 2026. Version 1.1, October 2026: corrected to document the prototype as built. Version 1.0 remains in the Zenodo version history.